Demo Presentation

Object-Oriented Programming

The Four Core Principles

Most. Sonia Khatun

B.Sc. in CSE
Bangladesh University of Engineering and Technology

What is Object-Oriented Programming (OOP)?

OOP is a programming paradigm that organizes software around **objects** that combine data and behavior.

An object contains:

- **State -** data or attributes
- **Behavior -** methods or operations

Simple idea

A **class** is a blueprint.
An **object** is an instance of that blueprint.

Example: Car State: color Behavior: start()

JAVA CODE

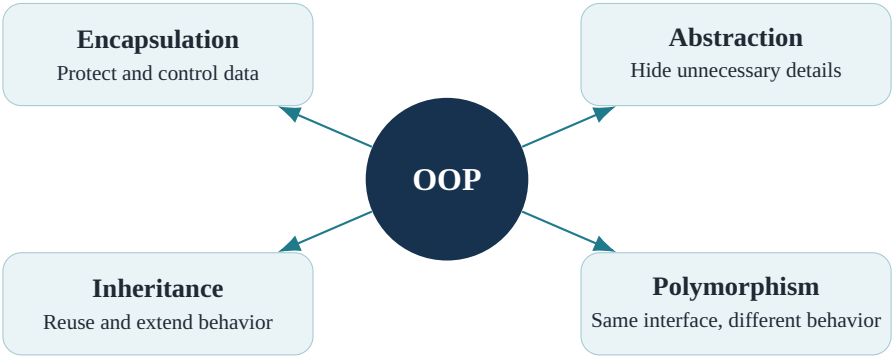
```java
class Car {
    private String color;

    Car(String color) {
        this.color = color;
    }

    void start() {
        System.out.println("Car started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Blue");
        myCar.start();
    }
}
```

The Four Core Principles of OOP



Together, these principles make software easier to organize, reuse, maintain, and extend.

1. Encapsulation - Protecting Data

Encapsulation means bundling data with the methods that operate on it, while controlling access to an object's internal state.

Why?

Without control, anyone could set an invalid value directly.

Bank account example:

- `balance` is `private`
- Changes happen through methods
- The class can validate every change

Key idea **Bundle state + control access**

JAVA CODE

```java
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(1000);
        // account.balance = -5000; // error: private
    }
}
```

2. Abstraction - Hiding Complexity

Abstraction means exposing essential behavior while hiding unnecessary implementation details.

Real-life analogy

A driver uses **start()** and **brake()** without knowing every internal engine mechanism.

In Java, abstraction can be achieved using:

- Abstract classes
- Interfaces

Key idea **Expose what; hide how**

JAVA CODE

```
abstract class Vehicle {
    abstract void move();

    void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    void move() {
        System.out.println("Car moves");
    }
}
```



3. Inheritance - Reusing and Extending Behavior

Inheritance allows a new class to reuse and extend accessible data and behavior from an existing class.

Relationship

Car is a Vehicle - this is an **IS-A** relationship.

Benefits:

- Code reuse
- Less duplication
- Easier extension



JAVA CODE

```java
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void openTrunk() {
        System.out.println("Trunk opened");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();      // inherited
        car.openTrunk();  // own behavior
    }
}
```

4. Polymorphism - One Interface, Many Behaviors

Polymorphism allows the same operation to produce different behavior depending on the actual object type.

Example

Every vehicle can `move()`, but each vehicle moves differently.



Key idea **Same operation, different behavior**

JAVA CODE

```java
class Vehicle {
    void move() { }
}

class Car extends Vehicle {
    @Override
    void move() {
        System.out.println("Car drives");
    }
}

class Boat extends Vehicle {
    @Override
    void move() {
        System.out.println("Boat sails");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.move();   // Car drives
    }
}
```

4.1 Compile-Time Polymorphism - Overloading

Compile-time polymorphism occurs when the compiler determines which method to call based on the method signature.

Method Overloading

The same method name is used with **different parameter lists**.

Overloaded methods need different **parameter lists**; return type alone is not enough.

Key idea **Same name, different signatures**

JAVA CODE

```java
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(2, 3));
        System.out.println(c.add(2.5, 3.5));
    }
}
```

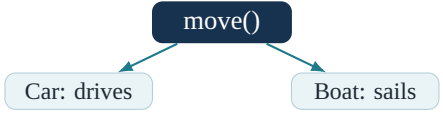
4.2 Runtime Polymorphism - Overriding

Runtime polymorphism occurs when the method to execute is determined at runtime according to the actual object type.

Method Overriding

A subclass provides its own implementation of an inherited method.

Every vehicle can `move()`, but each vehicle moves differently.



Key idea **Same call, behavior chosen at runtime**

JAVA CODE

```java
class Vehicle {
    void move() { }
}

class Car extends Vehicle {
    @Override
    void move() {
        System.out.println("Car drives");
    }
}

class Boat extends Vehicle {
    @Override
    void move() {
        System.out.println("Boat sails");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.move();   // Car drives
    }
}
```

Putting the Four Principles Together

Principle	Main Question	Core Idea
Encapsulation	How is internal state controlled?	Bundle data and behavior; control access
Abstraction	What details should the user see?	Expose essentials; hide implementation details
Inheritance	What can be reused and extended?	Derive new classes from existing classes
Polymorphism	Can one name/interface take multiple forms?	Overloading at compile time; overriding at runtime

OOP helps us model real-world systems through modular, reusable, and maintainable code.

Thank You